SE 2228 Analysis & Design of Algorithms, Yaşar University, 2025-2026 Fall

Lab 01: C Review of Pointers and Dynamic Memory

Mert Hazar Hasgül, mert.hasgul@yasar.edu.tr

Compile any file with:

```
TERMINAL
gcc -std=c11 -Wall -Wextra -pedantic file.c -o file
```

Contents

1. Pointers
   1.1 Value and address
   1.2 A pointer stores an address
   1.3 Pointer types and sizes
   1.4 Writing through a pointer
   1.5 Arrays and pointer arithmetic
   1.6 Call by value vs call by reference
   1.7 Double pointers and 2-D arrays
2. Dynamic Memory Allocation
   2.1 malloc
   2.2 calloc
   2.3 realloc
   2.4 free: the rules
3. Other Essentials
   3.1 chars and strings
   3.2 Function pointers and void *
   3.3 structs, nodes, and a dangling pointer
4. Pointer Confusions: 4 wrong ways and 1 right way
   bad 1: reassigning a pointer parameter
   bad 2: returning the address of a local
   bad 3: no memory behind the pointer
   bad 4: malloc into a copy
   good: malloc through a double pointer
   main
5. Cheat Sheet

1. Pointers

1.1 Value and address

- Every variable has a **value** and an **address** in memory.
- `&x` gives the address of `x`. `%p` prints an address.
- `scanf` needs `&x` because it must write *into* `x`.

`lab-intro/1-Pointers/1-ValueAndAddress.c`

```c
#include <stdio.h>

int main(void) {
    int var = 6;

    printf("value of var   : %d\n", var);
    printf("address of var : %p\n", (void *)&var); // %p prints an address

    // scanf needs an ADDRESS, because it writes INTO the variable
    int user_input;
    printf("Enter a value: ");
    if (1 != scanf("%d", &user_input)) {
        return 1;
    }
    printf("user_input     : %d (stored at %p)\n", user_input, (void *)&user_input);

    return 0;
}
```

Output, with the user typing `42` at the prompt:

```
OUTPUT
value of var   : 6
address of var : 00000049b29ff74c
Enter a value: 42
user_input     : 42 (stored at 00000049b29ff748)
```

> Addresses printed with `%p` change on every run. Values do not.

1.2 A pointer stores an address

- `int *p;` declares a pointer to an `int`. `int* p;` and `int * p;` mean the same.
- `p = &a;` stores the address of `a` in `p`.
- `*p` (dereference) reads the value at that address.

lab-intro/1-Pointers/2-PointerBasics.c

```c
#include <stdio.h>

int main(void) {
    int a = 10;
    int *p = &a; // p holds the address of a

    printf("*p = %d\n", *p);         // value p points to (10)
    printf(" p = %p\n", (void *)p);  // value of p: an address
    printf("&p = %p\n", (void *)&p); // address of p itself
    printf(" a = %d\n", a);          // value of a (10)
    printf("&a = %p\n", (void *)&a); // address of a: same as p

    return 0;
}
```

```
OUTPUT
*p = 10
 p = 000000c209dffc2c
&p = 000000c209dffc20
 a = 10
&a = 000000c209dffc2c
```

What memory looks like after this run:

Variable	Its address	Holds	Meaning
`p`	`...c20`	`...c2c`	address of `a`
`a`	`...c2c`	`10`	the value

- `p` holds `...c2c`. The value stored at `...c2c` is `10`. So `*p == a`.

1.3 Pointer types and sizes

- The **type** of a pointer says how many bytes to read at the address (`int *` reads 4, `double *` reads 8).
- The **size** of every pointer is the same: 4 bytes on 32-bit, 8 bytes on 64-bit.
- `void *` is a pointer with no type: it cannot be dereferenced until it is cast.

lab-intro/1-Pointers/3-PointerTypes.c

```c
#include <stdio.h>

int main(void) {
    printf("int      : %zu bytes\n", sizeof(int));
    printf("double   : %zu bytes\n", sizeof(double));
    printf("float    : %zu bytes\n", sizeof(float));
    printf("char     : %zu bytes\n", sizeof(char));

    printf("int *    : %zu bytes\n", sizeof(int *));
    printf("double * : %zu bytes\n", sizeof(double *));
    printf("float *  : %zu bytes\n", sizeof(float *));
    printf("char *   : %zu bytes\n", sizeof(char *));
    printf("void *   : %zu bytes\n", sizeof(void *));

    return 0;
}
```

```
OUTPUT
int      : 4 bytes
double   : 8 bytes
float    : 4 bytes
char     : 1 bytes
int *    : 8 bytes
double * : 8 bytes
float *  : 8 bytes
char *   : 8 bytes
void *   : 8 bytes
```

1.4 Writing through a pointer

- `*p = 20;` writes into the variable `p` points to.
- `a` and `*p` are **the same memory**: change one, the other follows.

`lab-intro/1-Pointers/4-WriteThroughPointer.c` (excerpt)

```c
    int a = 10;
    int *p = &a;

    printf("start     : a = %d, *p = %d\n", a, *p);

    *p = 20; // write through the pointer
    printf("*p = 20   : a = %d, *p = %d\n", a, *p);

    a = 30; // write the variable directly, p sees it too
    printf("a  = 30   : a = %d, *p = %d\n", a, *p);
```

```
OUTPUT
start     : a = 10, *p = 10
*p = 20   : a = 20, *p = 20
a  = 30   : a = 30, *p = 30
```

1.5 Arrays and pointer arithmetic

- An array name is the address of its first element: `notes == &notes[0]`.
- `notes + 1` moves by **one element** (`sizeof(int)` = 4 bytes), not one byte.
- `notes[i]` is just shorthand for `*(notes + i)`.

`lab-intro/1-Pointers/5-PointerArithmetic.c` (excerpt)

```c
    int notes[5] = {10, 20, 30, 40, 50};

    printf("\nnotes[1]     = %d\n", notes[1]);
    printf("*(notes + 1) = %d   (same element)\n", *(notes + 1));
    printf("notes        = %p\n", (void *)notes);
    printf("notes + 1    = %p   (+%zu bytes = one int)\n", (void *)(notes + 1), sizeof(notes[0]));
```

```
OUTPUT
notes[1]     = 20
*(notes + 1) = 20   (same element)
notes        = 0000008d6edff6e0
notes + 1    = 0000008d6edff6e4   (+4 bytes = one int)
```

1.6 Call by value vs call by reference

- C **always** passes arguments by value: the function gets a **copy**.
- To let a function change the caller's variable, pass its **address** and write through it.

`lab-intro/1-Pointers/6-CallByValue.c`

```c
#include <stdio.h>

void swap_by_value(int x, int y) { // x and y are copies of a and b
    int t = x;
    x = y;
    y = t;
    printf("inside swap    : x=%d y=%d\n", x, y);
}

void add_five(int count) {
    count += 5; // changes the copy only
}

int main(void) {
    int a = 10, b = 20;
    int calculation = 10;

    swap_by_value(a, b);
    printf("after swap     : a=%d b=%d\n", a, b);

    printf("before add_five: %d\n", calculation);
    add_five(calculation);
    printf("after add_five : %d\n", calculation);

    return 0;
}
```

```
OUTPUT
inside swap    : x=20 y=10
after swap     : a=10 b=20
before add_five: 10
after add_five : 10
```

`lab-intro/1-Pointers/7-CallByReference.c` is the same program. Only the functions and the calls change:

```c
void swap_by_reference(int *x, int *y) { // x and y point to a and b
    int t = *x;
    *x = *y;
    *y = t;
    printf("inside swap    : x=%d y=%d\n", *x, *y);
}

void add_five(int *count) {
    *count += 5; // changes the caller's variable
}

    // in main:
    swap_by_reference(&a, &b);
    add_five(&calculation);
```

```
OUTPUT
inside swap    : x=20 y=10
after swap     : a=20 b=10
before add_five: 10
after add_five : 15
```

1.7 Double pointers and 2-D arrays

- `int **double_ptr` stores the address of the pointer `ptr`. `**double_ptr` reaches the value in two hops.
- A 2-D array is reached the same way: `m[i][j]` is `*(*(m + i) + j)`.
- The parameter `int m[rows][cols]` is a C99 variable-length array: fine with gcc and clang, not supported by MSVC.

lab-intro/1-Pointers/8-DoublePointer.c

```c
#include <stdio.h>

void print_2d(int rows, int cols, int m[rows][cols]) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            printf("%3d", m[i][j]);
        }
        printf("\n");
    }
}

int main(void) {
    int var = 789;
    int *ptr = &var;         // ptr -> var
    int **double_ptr = &ptr; // double_ptr -> ptr -> var

    printf("var        = %-16d  &var        = %p\n", var, (void *)&var);
    printf("ptr        = %p  &ptr        = %p\n", (void *)ptr, (void *)&ptr);
    printf("double_ptr = %p  &double_ptr = %p\n", (void *)double_ptr, (void *)&double_ptr);
    printf("*ptr = %d, **double_ptr = %d\n", *ptr, **double_ptr);

    int m[3][4] = {{1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12}};
    int rows = sizeof(m) / sizeof(m[0]);       // 48 / 16 = 3
    int cols = sizeof(m[0]) / sizeof(m[0][0]); // 16 / 4  = 4
    printf("\nrows = %d, cols = %d\n", rows, cols);

    printf("index form m[i][j]:\n");
    print_2d(rows, cols, m);

    printf("pointer form *(*(m + i) + j):\n");
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            printf("%3d", *(*(m + i) + j));
        }
        printf("\n");
    }

    return 0;
}
```

```
OUTPUT
var        = 789               &var        = 000000b61b1ff77c
ptr        = 000000b61b1ff77c  &ptr        = 000000b61b1ff770
double_ptr = 000000b61b1ff770  &double_ptr = 000000b61b1ff768
*ptr = 789, **double_ptr = 789

rows = 3, cols = 4
index form m[i][j]:
  1  2  3  4
  5  6  7  8
  9 10 11 12
pointer form *(*(m + i) + j):
  1  2  3  4
  5  6  7  8
  9 10 11 12
```

Variable	Its address	Holds	Meaning
`double_ptr`	`...768`	`...770`	address of `ptr`
`ptr`	`...770`	`...77c`	address of `var`
`var`	`...77c`	`789`	the value

- Follow the chain: `double_ptr` holds the address of `ptr`, `ptr` holds the address of `var`, and `var` holds `789`.

2. Dynamic Memory Allocation

Four functions, all in `<stdlib.h>`:

Function	Does	Initialised?
`malloc(bytes)`	reserves a block on the heap	no (garbage)
`calloc(count, size)`	reserves `count × size` bytes	yes, all 0
`realloc(ptr, bytes)`	resizes a block, keeps old contents	new part: no
`free(ptr)`	releases a block	-

- All three allocators return `void *` (cast to the needed type) or **NULL** on failure. Always check.
- Local variables live on the **stack**: they appear at `{` and vanish at `}`.
- Heap memory lives until `free()`. Forgetting `free()` causes a **memory leak**.

2.1 malloc

`lab-intro/2-MemoryAllocation/1-Malloc.c` (excerpt: one value, then a 1-D array)

```c
    // 1) one int on the heap
    int *ip = (int *)malloc(sizeof(int));
    if (NULL == ip) {
        return 1;
    }
    *ip = 126;
    printf("1) one int   : ip = %p, *ip = %d\n", (void *)ip, *ip);
    free(ip);

    // 2) a 1-D array of 5 ints
    int *arr = (int *)malloc(5 * sizeof(int));
    if (NULL == arr) {
        return 1;
    }
    for (int i = 0; i < 5; i++) {
        arr[i] = i;
    }
    printf("2) 1-D array :");
    print_1d(arr, 5);
    free(arr);
```

A 2-D array on the heap is an array of row pointers. Each row is allocated separately:

```c
int **alloc_2d(int rows, int cols) { // NULL checks omitted to keep the example short
    int **arr = (int **)malloc(rows * sizeof(int *)); // array of row pointers
    for (int i = 0; i < rows; i++) {
        arr[i] = (int *)malloc(cols * sizeof(int)); // same as *(arr + i) = ...
    }
    return arr;
}

void free_2d(int **arr, int rows) {
    for (int i = 0; i < rows; i++) {
        free(arr[i]); // free every row first ...
    }
    free(arr); // ... then the array of row pointers
}

    // 3) in main: a 3x4 2-D array
    int **m = alloc_2d(3, 4);
    fill_2d(m, 3, 4);   // arr[i][j] = rand() % 20 + 1
    printf("3) 2-D array :\n");
    print_2d(m, 3, 4);
    free_2d(m, 3);
```

OUTPUT

```
1) one int   : ip = 0000014a2c2e7f30, *ip = 126
2) 1-D array :  0  1  2  3  4
3) 2-D array :
  2  8 15  1
 10  5 19 19
  3  5  6  6
```

MEMORY

```
m ──> [ m[0] ] ──> [  2  8 15  1 ]
      [ m[1] ] ──> [ 10  5 19 19 ]
      [ m[2] ] ──> [  3  5  6  6 ]
```

- Free in reverse order: rows first, then the pointer array. Freeing `m` alone leaks all rows.

2.2 calloc

- Same as `malloc`, but the block is **zeroed**.

lab-intro/2-MemoryAllocation/2-Calloc.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int *ip = (int *)calloc(5, sizeof(int)); // 5 ints, all zero
    if (NULL == ip) {
        return 1;
    }

    printf("right after calloc:\n");
    for (int i = 0; i < 5; i++) {
        printf("%p -> %d\n", (void *)(ip + i), *(ip + i));
    }

    for (int i = 0; i < 5; i++) {
        *(ip + i) = (i + 1) * 10; // same as ip[i] = ...
    }

    printf("after filling:\n");
    for (int i = 0; i < 5; i++) {
        printf("%p -> %d\n", (void *)(ip + i), *(ip + i));
    }

    free(ip);
    return 0;
}
```

OUTPUT

```
right after calloc:
0000011fc4fc7f50 -> 0
0000011fc4fc7f54 -> 0
0000011fc4fc7f58 -> 0
0000011fc4fc7f5c -> 0
0000011fc4fc7f60 -> 0
after filling:
0000011fc4fc7f50 -> 10
0000011fc4fc7f54 -> 20
0000011fc4fc7f58 -> 30
0000011fc4fc7f5c -> 40
0000011fc4fc7f60 -> 50
```

- Consecutive addresses differ by 4: the 5 ints are **contiguous**.

2.3 realloc

- Grows (or shrinks) an existing block. Old contents are kept.
- The block **may move**. Use a temporary pointer: if `realloc` fails, the old block is still yours to free.

lab-intro/2-MemoryAllocation/3-Realloc.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int *ip = (int *)malloc(5 * sizeof(int));
    if (NULL == ip) {
        return 1;
    }

    int id;
    for (id = 0; id < 5; id++) {
        *(ip + id) = id + 1;
        printf("%p -> %d\n", (void *)(ip + id), *(ip + id));
    }

    int *tmp = (int *)realloc(ip, 10 * sizeof(int)); // grow to 10 ints
    if (NULL == tmp) {
        free(ip); // on failure the old block is still valid
        return 1;
    }
    ip = tmp;

    printf("extended values (id continues from %d):\n", id);
    for (; id < 10; id++) {
        *(ip + id) = id + 1;
        printf("%p -> %d\n", (void *)(ip + id), *(ip + id));
    }

    free(ip);
    return 0;
}
```

```
OUTPUT
0000016cffe17f60 -> 1
0000016cffe17f64 -> 2
0000016cffe17f68 -> 3
0000016cffe17f6c -> 4
0000016cffe17f70 -> 5
extended values (id continues from 5):
0000016cffe17f74 -> 6
0000016cffe17f78 -> 7
0000016cffe17f7c -> 8
0000016cffe17f80 -> 9
0000016cffe17f84 -> 10
```

2.4 free: the rules

- Every `malloc` / `calloc` / `realloc` needs exactly **one** `free`.
- Never `free` stack memory (`int a; free(&a);`) and never `free` twice.
- After `free(p)`, `p` is **dangling**: it still holds the old address, but the memory behind it is gone. Set `p = NULL;` so that a later misuse crashes immediately instead of corrupting memory.

3. Other Essentials

3.1 chars and strings

- A string is a `char` array that **ends with** `'\0'`. `%s` prints until it finds `'\0'`.
- `char str[] = "ege"` is 4 bytes (copy, writable). `const char *ptr = "ege"` is a pointer to a read-only literal.
- `const char *names[3]` is an array of 3 pointers: an array of strings.

`lab-intro/3-Others/1-CharsStrings.c`

```c
#include <stdio.h>

int main(void) {
    char one_char = 'e';
    char char_arr[3] = {'e', 'g', 'e'};               // NOT a string: no '\0' at the end
    char str[] = "ege";                               // {'e','g','e','\0'}: 4 bytes
    const char *ptr = "ege";                          // pointer to a read-only literal
    const char *names[3] = {"yaren", "ege", "taylan"}; // array of strings

    printf("%c\n", one_char);

    for (int i = 0; i < 3; i++) {
        printf("%c", char_arr[i]); // print char by char
    }
    printf("\n");

    printf("%s\n", str); // the easy way
    printf("%s\n", ptr);
    printf("sizeof str = %zu, sizeof ptr = %zu\n", sizeof(str), sizeof(ptr));

    for (int i = 0; i < 3; i++) {
        printf("%s ", names[i]);
    }
    printf("\n");

    return 0;
}
```

```
OUTPUT
e
ege
ege
ege
sizeof str = 4, sizeof ptr = 8
yaren ege taylan
```

3.2 Function pointers and void *

- A function also has an address. `void (*fill)(int *, int)` is a pointer to a function taking `(int *, int)` and returning `void`.
- Passing a function as a parameter lets one routine (`general_fill`) run many behaviours.
- `void *` holds any address but has no type: **cast** it (`(int *)raw`) before indexing.

`lab-intro/3-Others/2-FunctionPointers.c` (excerpt; `fill_down` mirrors `fill_up`, `print_1d` prints an array)

```c
void fill_up(int *arr, int n) {
    for (int i = 0; i < n; i++) {
```

```c
        arr[i] = i;
    }
}

/* third parameter: a pointer to a function taking (int *, int), returning void */
void general_fill(int *arr, int n, void (*fill)(int *, int)) {
    fill(arr, n);
}

void copy_to_void(void *dst, const int *src, int n) {
    for (int i = 0; i < n; i++) {
        ((int *)dst)[i] = src[i]; // cast void * to int * before indexing
    }
}

void print_void_as_int(const void *arr, int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", ((const int *)arr)[i]);
    }
    printf("\n");
}

int main(void) {
    int a[5];

    general_fill(a, 5, fill_up); // pass the function itself
    print_1d(a, 5);
    general_fill(a, 5, fill_down);
    print_1d(a, 5);

    void (*fill)(int *, int) = fill_up; // a variable that holds a function
    fill(a, 5);
    print_1d(a, 5);

    void *raw = malloc(5 * sizeof(int)); // sizeof(void) is NOT valid C
    if (NULL == raw) {
        return 1;
    }
    copy_to_void(raw, a, 5);
    print_void_as_int(raw, 5);
    free(raw);

    return 0;
}
```

```
OUTPUT
0 1 2 3 4
5 4 3 2 1
0 1 2 3 4
0 1 2 3 4
```

3.3 structs, nodes, and a dangling pointer

- A `struct` groups fields. `n->x` is shorthand for `(*n).x`.
- A struct that contains a pointer to its own type is a **node**: the building block of linked lists.
- Reading through a pointer after `free` is **undefined behaviour**: you may see the old value, garbage, or a crash.

lab-intro/3-Others/3-StructNode.c

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct node {
    int *number;       // points to an int somewhere else
    struct node *next; // points to the next node, or NULL
} Node;

Node *node_init(void) {
    Node *n = (Node *)malloc(sizeof(Node));
    if (NULL == n) {
        return NULL;
    }
    n->number = NULL; // n->x is short for (*n).x
    n->next = NULL;
    return n;
}

int main(void) {
    int first = 5, second = 10;

    Node *head = node_init();
    Node *tail = node_init();
    head->number = &first;
    head->next = tail; // head -> tail -> NULL
    tail->number = &second;

    printf("head       : number at %p = %d\n", (void *)head->number, *head->number);
    printf("head->next : number at %p = %d\n", (void *)head->next->number, *head->next->number);

    free(head);
    /* WHY DO WE (OFTEN) STILL SEE THE VALUE?
       head is now a DANGLING pointer. Reading through it is undefined behaviour:
       the bytes may still be there, or the program may crash. Never rely on it. */
    printf("after free : head->next->number = %d  (undefined behaviour!)\n", *head->next->number);
    head = NULL; // the safe habit: NULL after free

    free(tail);
    return 0;
}
```

OUTPUT

```
head       : number at 00000002011ffdcc = 5
head->next : number at 00000002011ffdc8 = 10
after free : head->next->number = -1960542304  (undefined behaviour!)
```

- On this run the freed block was already reused, so we got garbage. On another machine you may still see `10`. Both are possible with undefined behaviour: **there is no rule**.

4. Pointer Confusions: 4 wrong ways and 1 right way

Goal: a function must give `main` a **new** `int` on the heap holding `35`. In `main`, `var` starts as `NULL` before every attempt.

Case	Parameter	What the function does	What `main` sees afterwards
bad 1	`int *p`	points its own copy at a local `x`	`var` is still `NULL`; reading `*var` crashes
bad 2	`int **p`	points `var` at a local `x`	`var` points to dead stack memory; reads garbage
bad 3	`int *p`	nothing: there is no memory behind `p`	`var` is still `NULL`; reading `*var` crashes
bad 4	`int *p`	mallocs into its own copy	`var` is still `NULL`; reading crashes, block leaked
good	`int **p`	mallocs into `*p`, which is `var`	`var` points to a valid heap `int`

Two rules decide every case:

1. To change a pointer inside a function, pass its address: `int **`.
2. Memory handed back to the caller must outlive the function: heap, never a local variable.

How to run: `main` has five blocks, each marked with start and end comment lines, and every crash line is active. A full run stops at the first crash. To show one case, comment out the other four blocks, rebuild, run. Each output panel below is the run with only that block active. A crash shows as exit code `0xC0000005` on Windows and as "Segmentation fault" on Linux.

`lab-intro/1-Pointers/9-Confusions.c`

bad 1: reassigning a pointer parameter

```c
void reassign_bad_1(int *p) { // p is a COPY of the caller's pointer
    int x = 123; // stack variable: lives only inside these braces
    p = &x;      // reassigns only the local copy
    printf("reassign_bad_1: addr: %p\n", (void *)p);
    printf("reassign_bad_1: val: %d\n", *p);
}
```

```
OUTPUT
reassign_bad_1: addr: 000000f0bc7ffc0c
reassign_bad_1: val: 123
main: after reassign_bad_1 addr: 0000000000000000
```

- The pointer itself is passed **by value**, so `var` is still `NULL` and the read of `*var` crashes.

bad 2: returning the address of a local

```c
void reassign_bad_2(int **p) {
    int x = 123;
    /* x is a stack variable. When this function returns, its memory may be reused by
       someone else. The caller receives a DANGLING pointer: reading it is undefined
       behaviour and may crash with a segfault. */
    *p = &x;
    printf("reassign_bad_2: addr: %p\n", (void *)*p);
    printf("reassign_bad_2: val: %d\n", **p);
}
```

```
OUTPUT
reassign_bad_2: addr: 000000f182dffacc
reassign_bad_2: val: 123
main: after reassign_bad_2 addr: 000000f182dffacc
main: after reassign_bad_2 val: 0
```

- The double pointer works and `var` did change, but it now points to a **dead stack frame**.
- No crash. `main` printed `0`, not `123`: the stack slot was already reused. Undefined behaviour that looks like it works is the dangerous kind.

bad 3: no memory behind the pointer

```c
void reassign_bad_3(int *p, int x) {
    // p is NULL: there is no memory behind it, nothing was malloc'ed.
    // *p = x;  <- writing to address NULL: segmentation fault!
    printf("reassign_bad_3: addr: %p (nowhere to store %d)\n", (void *)p, x);
}
```

```
OUTPUT
reassign_bad_3: addr: 0000000000000000 (nowhere to store 35)
```

- A pointer is not storage by itself: the read of `*var` crashes, and the commented-out `*p = x;` inside the function would crash the same way.

bad 4: malloc into a copy

```c
int reassign_bad_4(int *p, int x) { // returns int, not void: check return values!
    p = (int *)malloc(sizeof(int)); // changes only the LOCAL copy: pointers are passed by value
    if (NULL == p) { // do not use (var == value), always use (value == var)
        return -1;
    }

    *p = x; // this will not persist!
    printf("reassign_bad_4: addr: %p\n", (void *)p);
    printf("reassign_bad_4: val: %d\n", *p);

    // the caller never receives this pointer, so the block can never be freed: MEMORY LEAK
    return 0;
}
```

```
OUTPUT
reassign_bad_4: addr: 000001ed29cb7dc0
reassign_bad_4: val: 35
main: after reassign_bad_4 addr: 0000000000000000
```

- Same mistake as bad 1, so the read of `*var` crashes again, plus a **leak**: that block can never be freed.
- Note `NULL == p` instead of `p == NULL`. If you type `=` instead of `==` by mistake, the code fails to compile instead of silently assigning.

good: malloc through a double pointer

```
int reassign_good(int **p, int x) { // changes the pointer itself via a double pointer!
    *p = (int *)malloc(sizeof(int));
    if (NULL == *p) { // do not use (var == value), always use (value == var)
        return -1;
    }

    **p = x;
    printf("reassign_good: addr: %p\n", (void *)*p);
    printf("reassign_good: val: %d\n", **p);

    return 0;
}
```

```
OUTPUT
reassign_good: addr: 0000029bf07f7dc0
reassign_good: val: 35
main: after reassign_good addr: 0000029bf07f7dc0
main: after reassign_good val: 35
```

- Rule 1 in action: `&var` lets the function change `var` itself, call by reference applied to the pointer.
- This is the only block that hands `main` usable memory, so it is the only one that ends with `free`.

main

```
int main(void) {
    int *var = NULL;

    // ========== bad 1: start ==========
    reassign_bad_1(var);
    printf("main: after reassign_bad_1 addr: %p\n", (void *)var); // still NULL
    printf("main: after reassign_bad_1 val: %d\n", *var);         // segfault: dereferencing NULL
    printf("\n");
    // ========== bad 1: end ============

    // ========== bad 2: start ==========
    var = NULL;
    reassign_bad_2(&var);
    printf("main: after reassign_bad_2 addr: %p\n", (void *)var);
    printf("main: after reassign_bad_2 val: %d\n", *var);         // undefined: dead stack memory
    printf("\n");
    // ========== bad 2: end ============

    // ========== bad 3: start ==========
    var = NULL;
    reassign_bad_3(var, NEW_VAL);
    printf("main: after reassign_bad_3 val: %d\n", *var);         // segfault: var is NULL
    printf("\n");
    // ========== bad 3: end ============

    // ========== bad 4: start ==========
    var = NULL;
    if (0 != reassign_bad_4(var, NEW_VAL)) {
        return -1;
```

```
    }
    printf("main: after reassign_bad_4 addr: %p\n", (void *)var);
    printf("main: after reassign_bad_4 val: %d\n", *var);         // segfault: var is still NULL
    printf("\n");
    // ========== bad 4: end ============

    // ========== good: start ===========
    var = NULL;
    if (0 != reassign_good(&var, NEW_VAL)) {
        return -1;
    }
    printf("main: after reassign_good addr: %p\n", (void *)var);
    printf("main: after reassign_good val: %d\n", *var);
    printf("\n");

    free(var);
    var = NULL; // set NULL after free()
    // ========== good: end =============

    return 0;
}
```

Run as delivered, with all five blocks active, it stops at the first read of `*var`, inside bad 1:

```
OUTPUT
reassign_bad_1: addr: 00000056f63ffc9c
reassign_bad_1: val: 123
main: after reassign_bad_1 addr: 0000000000000000
```

5. Cheat Sheet

Syntax	Meaning
`int *p;`	`p` is a pointer to an `int`
`&a`	address of `a`
`*p`	value at the address in `p`
`p = &a;`	`p` now points to `a`
`*p = 5;`	write `5` into `a`
`int **double_ptr;`	pointer to a pointer; `**double_ptr` is the value
`a[i]`	`*(a + i)`
`m[i][j]`	`*(*(m + i) + j)`
`n->x`	`(*n).x`
`(int *)v`	give a `void *` a type
`NULL`	points to nothing; never dereference it

Want to change ...	Pass ...	Inside the function
the caller's `int a`	`&a` (`int *`)	`*p = ...`
the caller's `int *ptr`	`&ptr` (`int **`)	`*p = malloc(...)`

Habits that prevent most bugs:

1. Check every allocation: `if (NULL == p) { ... }`
2. One `free` per allocation. Free the rows before the row array.
3. `p = NULL;` after `free(p);`
4. Never return or store the address of a local variable.
5. Compile with `-Wall -Wextra -pedantic` and fix every warning.